

7. Un nombre à l'écran

`texte` écrit des lettres ; `nombre` écrit un calcul.

1. Ce qu'il faut comprendre

`texte()` ne sait écrire que ce qui est entre guillemets, décidé à la compilation. **Un score qui change veut** `nombre()`.

`nombre(colonne, ligne, valeur)` écrit en base dix, sur **trois chiffres par défaut** — `042`. Un quatrième argument dit combien on en veut : `nombre(8, 4, vies, 1)` écrit `3` tout seul.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1328 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    uint8_t vies = 3;
    uint8_t score = 42;

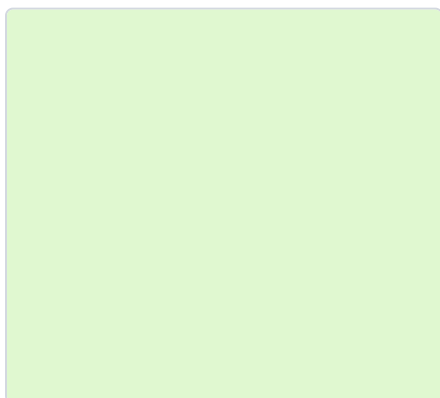
    texte(2, 4, "VIES");
    nombre(8, 4, vies, 1);

    texte(2, 6, "SCORE");
    nombre(8, 6, score);

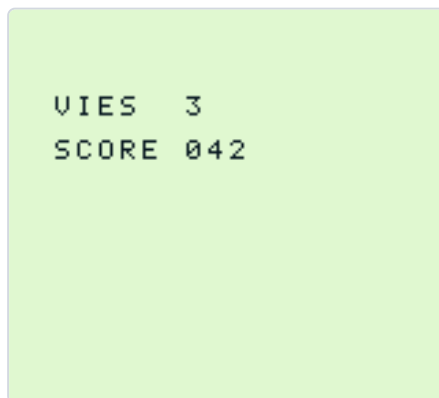
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

« VIES 3 » et « SCORE 042 ».

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ un seul chiffre quand on le demande
- ✓ trois chiffres par défaut, zéros compris
- ✓ les étiquettes sont là

Ensuite : **8. Effacer sans compter les lettres** — niveau 1 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++